# A#2: Markov Random Field (Triangle Network)

## Implementation Restrictions

**Minimum number of students 5**

**Maximum number of students 6**

**Students must follow these rules strictly:**

- **No Built-in Shortcuts for Computation**

  **You are NOT allowed to use functions that automatically perform:**

  - **Probability calculations**
  - **Normalization utilities**
  - **Marginalization functions**
  - **Predefined combinatorics tools**

- **No Hardcoding Results**

  - **Do NOT manually write the joint probability table**
  - **Do NOT directly assign final probability values**
  - **All results must be computed from the factor functions**

- **Required Implementation Approach**

  **You MUST:**

  - **Compute the joint distribution using factor multiplication**
  - **Compute the partition function Z using your own code from scratch**
  - **Normalize probabilities using your own code**
  - **Implement query and marginal functions explicitly**

- **Your code must clearly show:**

  - **How each probability is computed**
  - **How values are accumulated**
  - **How normalization is performed**

### No Copy-Paste Solutions

- **You are expected to write your own implementation**
- **Using pre-written code from the internet or AI BOTS is not allowed**

### Penalty Rules

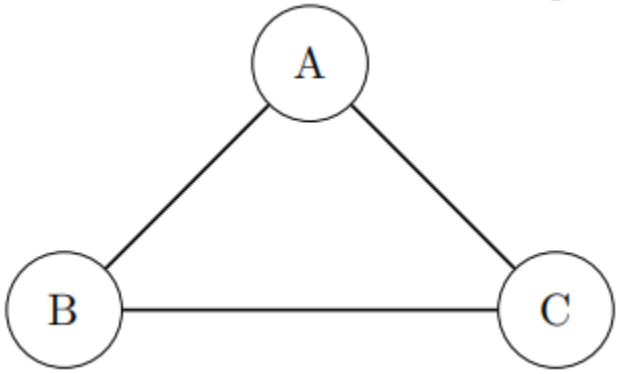
- **Using forbidden libraries → 0 marks for implementation**
- **Hardcoded results → 0 marks for correctness**
- **Missing query or marginal functions → grade reduction**

## Objective

In this assignment, you will implement a **Markov Random Field (MRF)** with three binary variables and compute probabilities from given clique potentials.

Consider three binary variables:

A, B, C ∈ {0,1}




The network has a circular structure with three edges forming the maximal cliques {A, B}, {B, C}, and {C, A}.

The clique potentials (factors) are defined as follows:

φ1(A, B) = 4 if A = B, and 2 if A = B.

φ2(B, C) = 5 if B = C, and 1 if B = C.

φ3(C, A) = 3 if C = A, and 1 if C = A.

**Task 1: Implement the Factors**

Write Python functions for each factor:

```
def phi1(a, b):
    pass

def phi2(b, c):
    pass

def phi3(c, a):
    pass
```

**Task 2: Construct the Joint Distribution (Unnormalized)**

Compute the unnormalized joint distribution:

$$\tilde{P}(A, B, C) = \varphi_1(A, B) \times \varphi_2(B, C) \times \varphi_3(C, A)$$

- Create Generate all combinations function of (A, B, C)
- Store results in a dictionary

```
def all_comp(a,b, c):
    #should return the truth table of all combinations, each one with its
```

own $\tilde{P}(A, B, C) = \varphi_1(A, B) \times \varphi_2(B, C) \times \varphi_3(C, A)$

```
    pass
```

**Task 3: Compute the Partition Function**

- Create a function that returns : $Z = \sum_{A,B,C} \tilde{P}(A, B, C)$

**Task 4: Normalize the Distribution**

Compute the normalized probability:

- Create a function that returns all the normalized probabilities table :

$$P(A,B,C) = \frac{\tilde{P}(A,B,C)}{Z}$$

Example of the o/p

| | | | | |
|---|---|---|---|---|
| A0 | B0 | C0 | Q1.Q2.Q3 | P(A0,B0,C0) |
| A0 | B0 | C1 | Q1.Q2.Q3 | P(A0,B0,C1) |
| A0 | B1 | C0 | Q1.Q2.Q3 | P(A0,B1,C0) |
| A0 | B1 | C1 | Q1.Q2.Q3 | P(A0,B1,C1) |
| A1 | B0 | C0 | Q1.Q2.Q3 | P(A1,B0,C0) |
| A1 | B0 | C1 | Q1.Q2.Q3 | P(A1,B0,C1) |
| A1 | B1 | C0 | Q1.Q2.Q3 | P(A1,B1,C0) |
| A1 | B1 | C1 | Q1.Q2.Q3 | P(A1,B1,C1) |

**Task 5: General Probability Query Function**

Create a function that returns the probability of **any configuration**:

```
def query(a, b, c):
    pass
```

Example usage:

```
print(query(1,1,1))
print(query(0,1,0))
```

**Task 6: Marginal Probability Function**

Create a function to compute marginal probabilities:

```
def marginal(variable, value):
    pass
```

Where:

- variable ∈ {'A', 'B', 'C'}
- value ∈ {0,1}

Example usage:

```
print(marginal('A', 1))
print(marginal('C', 0))
```